

QoS-Aware Routing Mechanism in OpenFlow-enabled Wireless Multimedia Sensor Networks

Li Han¹, Shimin Sun², Bokgyu Joo³, Xianshu Jin⁴ and Sunyoung Han⁴

1 School of Computer and Communication Engineering, Tianjin University of Technology,
Tianjin 300384, China

2 School of Computer Science & Software Engineering, Tianjin Polytechnic University,
Tianjin 300387, China

3 Department of Computer and Information Communication, Hongik University, Sejong
Campus, Sejong 337-701, Republic of Korea

4 Department of Computer Science and Engineering, Konkuk University, Seoul 143-701,
Republic of Korea

Correspondence should be addressed to Sunyoung Han; syhan@cclab.konkuk.ac.kr

Abstract

Applications supporting various multimedia data in wireless multimedia sensor networks (WMSNs) have specific QoS requirements on bandwidth, delay and/or packet loss. Perception of applications' QoS requirements and detection of link states are indispensable for the design of QoS-aware routing mechanism. Software defined networking (SDN) is suitable for the purpose since it provides visibility of network resources and programmable interfaces. OpenFlow is the most recognized realization of SDN.

We propose a QoS-aware routing mechanism for OpenFlow-enabled WMSNs. The mechanism consists of a framework and routing algorithms on SDN controller. The framework includes two functions: detection of link states among OpenFlow-enabled nodes and determination of flow's QoS requirements. The routing algorithms are achieved in two steps. First, it seeks the feasible paths that satisfy QoS requirements of a flow. If there is no path which satisfies the required QoS, the path will be decided by the proposed algorithms depending on flow types: delay-sensitive, bandwidth-sensitive, and best-effort traffic. We conducted experiments on a SDN testbed to evaluate our mechanism and compared the results with conventional routing protocols. The results show that proposed routing mechanism increases the throughput by 43% for video data and reduces the delay more than 30% for audio data.

Key words: Software defined networking, QoS-aware routing, OpenFlow, Wireless multimedia sensor networks.

1. Introduction

Recent advances in cost-efficient hardware technology have set new stage for the deployment of Wireless Sensor Networks (WSNs) with powerful sensor nodes in reasonable

cost. The paradigm is changing from scalar services (temperature, humidity, etc.) to real-time multimedia applications (video, audio, imaging, etc.). Examples of such Wireless Multimedia Sensor Network (WMSN) [1] applications include target tracking, environmental video/imaging surveillance, telemedicine, video vehicle detection, and battlefield intelligence [2-5].

Real-time multimedia applications usually require QoS assurances on bandwidth or delay but are tolerant to packet loss, unlike reliability requirement of scalar sensor data. High bandwidth and low delay requirements are major challenges for WMSNs and efficient QoS routing mechanisms are urgently needed to realize envisioned multimedia applications. Many QoS routing mechanisms and protocols [1] are proposed on optimizing network resource utilization, avoiding traffic congestion, and balancing network traffic. Most of them focus on calculating the optimal path with best network conditions without considering QoS requirements of applications. An application may consist of various types of flows, such as video, voice, text, and sensing data. Path decision could be made for each flow according to its QoS requirements and network conditions, which is conducive to network resource orchestration and QoS guarantee.

Software Defined Networking (SDN) [6] paradigm seems to be promising to present better network resource management, traffic control, and application classification in order to design an efficient QoS-aware routing mechanism. SDN separates the network into data plane and control plane, and introduces centralized controllers to abstract control functions for the network. Programmable APIs in SDN controllers (Floodlight, OpenDaylight, Ryu, etc.) [7] are especially suitable to realize QoS-aware routing. As the most recognized realization of SDN, OpenFlow [8] has many in-built functions such as traffic statistics collection, access control and network configuration. Some explorations on fusion of SDN and WSN have been carried out, such as Flow-Sensor [9], Sensor OpenFlow [10] and Software Defined Wireless Network (SDWN) [11]. There are still many issues need to be addressed, such as sensor network topology discovery, efficient self-organized routing mechanism, and network resource management.

In this paper, we propose a routing mechanism for WMSNs based on SDN technology. In the network architecture, OpenFlow-enabled nodes (OFNs) act as cluster headers, maintain connections with neighbor OFNs, and forward packets on behalf of sensor nodes. The mechanism consists of a QoS-aware routing framework and algorithms. The framework is constructed at the application layer of SDN controller with five components cooperated to achieve the proposed mechanism.

The rest of this paper is organized as follows. In Section 2, we discuss related studies on QoS routing in WSNs as well as the exploration of SDN in WSNs. In Section 3, we propose a QoS-aware routing mechanism with algorithms in detail. In Section 4, we evaluate the performance of our mechanism with experimental results. Finally, we summarize our experimental results and point out the future research directions in Section 5.

2. Related Works

QoS routing mechanisms in WSNs employ QoS metric-based path decisions. For multimedia data transmission, bandwidth, delay, packet loss, and energy efficiency are usually

taken as the major concern in QoS metrics. Collaborative QoS Routing (CQR) [12] was proposed to achieve efficient bandwidth utilization for real-time flows. Based on Reliable Energy Efficient routing Protocol (REEP) [13], Multimedia REEP (MREEP) [14] was designed as constraint-based routing for real-time and non-real-time traffic. Extended from SPEED [15], Power Adaptive SPEED (PASPEED) protocol [16] achieves energy efficiency by satisfying application-specific delay with minimum transmission cost. PASPEED selects the most energy efficient node among possible next-hop to provide the demanded delivery speed. For nodes with multiple network interfaces, if network capability cannot fulfill on-going flow's QoS requirements, the node can use other interfaces to transmit data for better QoS assurances [17].

Many QoS routing mechanisms have been presented for cluster-based WSNs. Cluster-Based Real-Time Protocol (CBRP-L) [18] combined LEACH [19] and CBRP [20], which discards high delay links and running LEACH to form clusters. The complexity of CBRP-L on uneven grid-based clustering and header selection algorithms result in excessive power consumption. Cluster-based ASARC [21] actuated multimedia sensors to maximize event information and minimum level of redundancy. Some mechanisms focused on suppression of message flooding and data aggregation to reserve network resource. GRMR [22] proposed a data centric routing mechanism including efficient regional multicast tree construction and data aggregation at the border node to offer QoS assurances.

Some explorations have been conducted in the area of extending SDN to WSNs. Sensor OpenFlow [10] proposed a Software-Defined WSN architecture and tried to address some key challenges, such as flow creation, in-network processing, backward and peer compatibility. A novel SDN-based WSN architecture was introduced in [23], where master node acts as the SDN controller and center nodes are OFNs. Center nodes request master node for the route of the first received packet of a flow. Then the master node calculates the route based on network topology and network conditions. SDWN [11] was presented as an extension of WSN, including the main idea, on-going researches and open issues. OpenRoads [24] envisions a world where users can move freely among SDWNs.

Although above literatures have better performance in some specific scenarios than compared mechanisms, they are still suffering from some shortcomings. Conventional QoS routing mechanisms mostly attempt to select the path with best network conditions without considering the flow's QoS requirements. For cluster-based QoS routing mechanisms, the election of cluster header and clustering process are energy consuming. SDN-enabled routing mechanisms currently are still the initial step and far from the completion. This paper focuses on the satisfaction of flow's QoS requirements, in order to guarantee the QoS of data delivery in resource-constrained OpenFlow-enabled WMSNs.

3. QoS-Aware Routing in OpenFlow-enabled WMSNs

In WSNs, data is transmitted through multihop nodes to destination commonly. Some of the intermediate nodes are powerful enough to endow with important roles, such as clustering and data aggregation. In this paper, we assume that some intermediate nodes are OpenFlow-enabled nodes (OFNs). Each OFN maintains a control interface and several flow tables as defined in OpenFlow protocol [8]. Control interface is responsible for OpenFlow

message exchange with SDN controller. Flow tables store flow-oriented forwarding rules issued by controller. Controller is the core network component, which maintains a centralized control plane to manage all OFNs of the network. We present a QoS-aware routing mechanism for OpenFlow-enabled WMSNs, including routing framework and flow-based QoS-aware routing algorithms. This mechanism focuses on link selection among OFNs to provide optimal QoS guarantee and network resource utilization.

3.1 Network Structure and Topology Discovery

As illustrated in Fig. 1, the multihop sensor network consists of sensor nodes, a sink node at the bottom right corner, OFNs acting as cluster headers, and an SDN controller. Sensor nodes are only aware of the route to its cluster header (OFN). The controller maintains an overlay network among OFNs through multihop connections. All sensor nodes are clustered to their nearest OFN in hop-count. OFNs are elected as cluster headers automatically and maintains the connection with its neighbor OFNs. All traffic in a cluster is converged to OFNs before injected to the network. OFNs forward the traffic according to its flow table rules or ask the controller for action. This architecture simplifies access control, network management, and data aggregation.

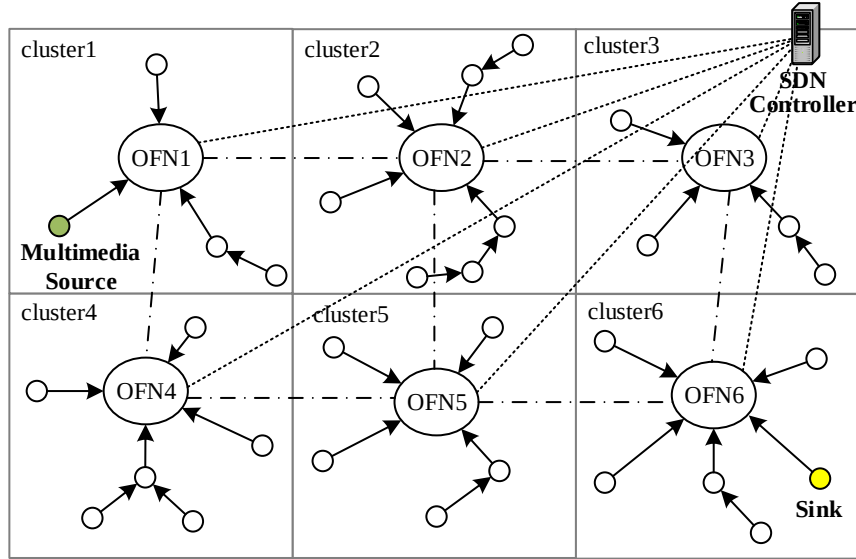


Figure 1 OpenFlow-enabled WMSN structure

To build the overlay network composed by OFNs, topology discovery should be performed firstly. Link Layer Discovery Protocol (LLDP) [25] is the inherent network topology discovery protocol in OpenFlow, which is a Layer 2 protocol to advertise and receive information from its adjacent OFNs. LLDP was designed for single-hop link discovery between OFNs. It is hard to directly apply to WSNs since most of the sensor nodes are not OpenFlow-enabled due to resource constraints. For hybrid OpenFlow networks, Broadcast Domain Discovery Protocol (BDDP) [25] was derived from LLDP and had been programmed in some open source controllers, such as Floodlight and OpenDayLight. BDDP has same packet format with LLDP, but destination MAC address is changed from multicast

address to broadcast address (ff:ff:ff:ff:ff:ff). Therefore, BDDP messages can be penetrated through OpenFlow indirect links.

The problem is that ad-hoc feature of WSN may result in BDDP message diffusing to the whole WSN, which leads to traffic overhead and energy inefficiency. Therefore, it is infeasible to apply BDDP directly for topology discovery in WSNs. To avoid flooding of BDDP messages, we propose gradual topology discovery mechanism. This mechanism limits the relay hops of BDDP messages by managing the Time-to-Live (TTL) field.

In initial phase, controller sends a BDDP message (TTL value=1) encapsulated in *Packet-Out* message to all OFNs in the network. After decapsulating the received *Packet-Out* message, OFNs advertise the BDDP message to its neighbor nodes. The neighbor nodes could be OFNs or conventional sensor nodes. If the neighbor is an OFN, it encapsulates this BDDP message in *Packet-In* message and sends it to controller. If not, it will notice that the TTL value is 1 and discard the message. After that, the controller gathers all the one-hop links among OFNs. Subsequently, the controller injects a new BDDP message with increased TTL value to the network to find the collection of routes with multihop links among OFNs. This process is completed until all OFNs have at least one route with other OFNs. Not only the shortest route but also multiple routes could be collected between each two OFNs. Finally, the route collection can be used to draw a network graph which includes all the OFNs. Optimal routes with best QoS conditions among OFNs are selected as the edges in the graph. Controller adjusts the graph dynamically according to link events. For example, edges among OFNs can be changed when a link goes down or a new link comes up.

3.2 Routing Framework

SDN architecture [26] consists of three layers: forwarding layer, control layer, and application layer, as shown in Fig. 2. The forwarding layer is made up of physical OFNs, which performs data forwarding according to the rules in flow tables. Control layer is responsible for the implementation of routing decision, topology discovery, traffic classification, and network configuration.

The proposed framework, as illustrated in Fig. 2, consists of five components: Link State Detection, Flow Classification, Flow Management, QoS-Aware Routing, and Per-flow Routing Policy. Those components work together based on the information collected from the control layer, such as topology and flow statistics. Link State Detection is responsible for detecting up-to-date link states among OFNs, including delay and bandwidth, using traffic monitoring tool sFlow [27] or NetFlow [28]. Flow Classification classifies online traffic [29] based on analyzing the flow statistics information using data mining technique, such as K-medoids algorithm [30]. Flow Management maintains flows' QoS requirements. QoS-Aware Routing is in charge of path determination depending on link states and flow's QoS requirements. Finally, routing decision is issued to control layer by Per-flow Routing Policy and distributed to OFNs located on the delivery path.

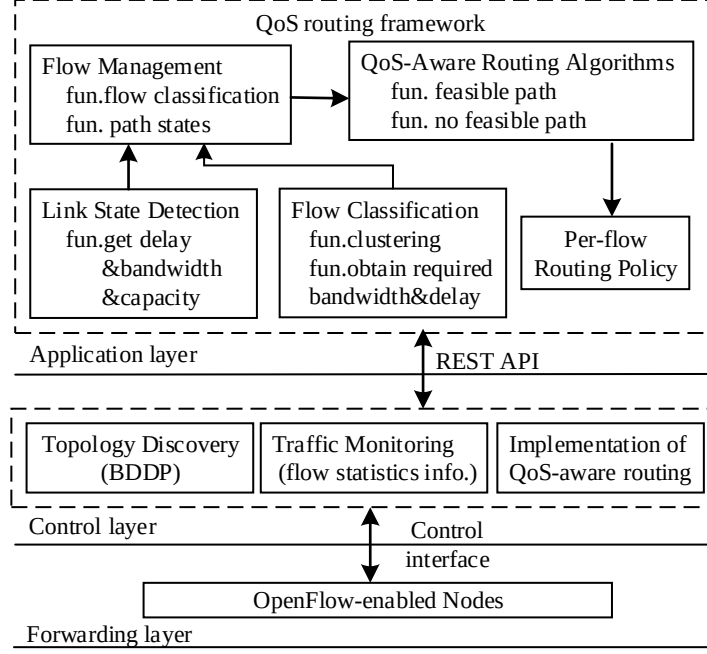


Figure 2 SDN architecture for QoS-aware routing mechanism

The routing modeling can be explained as follows. An OpenFlow-enabled WSN can be modeled as a network graph $G = (OFNs, E)$, where $OFNs$ represents OpenFlow-enabled nodes and $E = (e_1, e_2, e_3, \dots, e_n)$ is the link set among OFNs. Link states of edge e_i are indicated by vector: (c, d, u) , where ' c ' is the link capacity, ' d ' is the link delay and ' u ' is the link bandwidth utilization rate. For a given source-destination pair (s, t) , controller lists out all available paths with respective QoS metrics. Delay of a path P (d_p) is an additive metric while capacity c_p is an extremum metric. States of each path can be represented by the vectors (d_p, c_p) according to Eq. (1) and Eq. (2).

$$d_p = \sum_{i=1}^n d_{e_i}, \quad (1)$$

$$c_p = \min_{e_i \in p} c_{e_i}, \quad (2)$$

The utilization rate of a link can be calculated by Eq. (3). Here, b_{u_i} is the bandwidth usage can be obtained from sFlow by monitoring the traffic of the link.

$$u_{e_i} = \frac{b_{u_i}}{c_{e_i}}, \quad (3)$$

The available bandwidth of each path can be computed by Eq. (4):

$$b_a = \min_{1 \leq i \leq n} (c_{e_i} - b_{u_i}), \quad (4)$$

Our mechanism attempts to avoid traffic congestion when new flow is added to the link. Cost of a link is calculated according to link capacity (c_{e_i}), link usage (b_{u_i}), and required bandwidth (b_{req}) of the new flow. Eq. (5) is derived from the M/M/1 formula [31]. We consider not only the used bandwidth but also the required bandwidth of the new flow. Bandwidth cost function for each path is defined as a piecewise-linear approximation of the M/M/1 formula. This formula ensures that the link with lower utilization rate has smaller cost value. when link utilization approaches the link capacity, it becomes expensive to use this

link.

$$\varphi(b_{u,i} + b_{req}, c_{e,i}) = \begin{cases} 1, & 0 \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < \frac{1}{3} \\ 3, & \frac{1}{3} \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < \frac{2}{3} \\ 10, & \frac{2}{3} \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < \frac{9}{10} \\ 70, & \frac{9}{10} \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < 1 \\ 500, & 1 \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < \frac{11}{10} \\ 5000, & \frac{11}{10} \leq \frac{b_{u,i} + b_{req}}{c_{e,i}} < \infty \end{cases}, \quad (5)$$

The optimal path is the one with minimum bandwidth cost presented by sum function of each link cost (Eq. (6)).

$$cost_{bp} = \sum_{i=1}^n \varphi(b_{u,i} + b_{req}, c_{e,i}). \quad (6)$$

3.3 Flow-Based QoS-Aware Routing Algorithms

The routing algorithms are committed to find the most feasible path for specific QoS requirements. A feasible path is the one that can provide sufficient resource to satisfy all QoS requirements of the flow. For conventional multi-constrained QoS routing algorithms, if there is no path satisfying QoS constraints, the flow would be simply dropped. However, in many scenarios of WMSNs, such as emergence scene monitoring, it is helpful to receive real-time multimedia data as much as possible regardless of packet loss rate. Therefore, in the case of feasible path cannot be found, the proposed algorithms give the flow an optimal path which provides best-effort on QoS satisfaction.

The algorithm is divided into two steps. The first step is to find feasible paths which can assure flow's QoS requirements, while the second step is to find a best-effort path when feasible path doesn't exist. If feasible paths exist, one of the paths will be selected to transmit the flow. In the situation that multimedia data transmission, it could be difficult to find a feasible path in resource-constraint sensor networks. If feasible path doesn't exist, it means that flow's QoS requirements cannot be guaranteed during data transmission. Then, the routing algorithm is differentiated depending on flow types: bandwidth-sensitive traffic, delay-sensitive traffic, and best-effort traffic. This algorithm is to supply QoS assurances to the flow as much as possible. The detailed procedure is explained in Fig. 3.

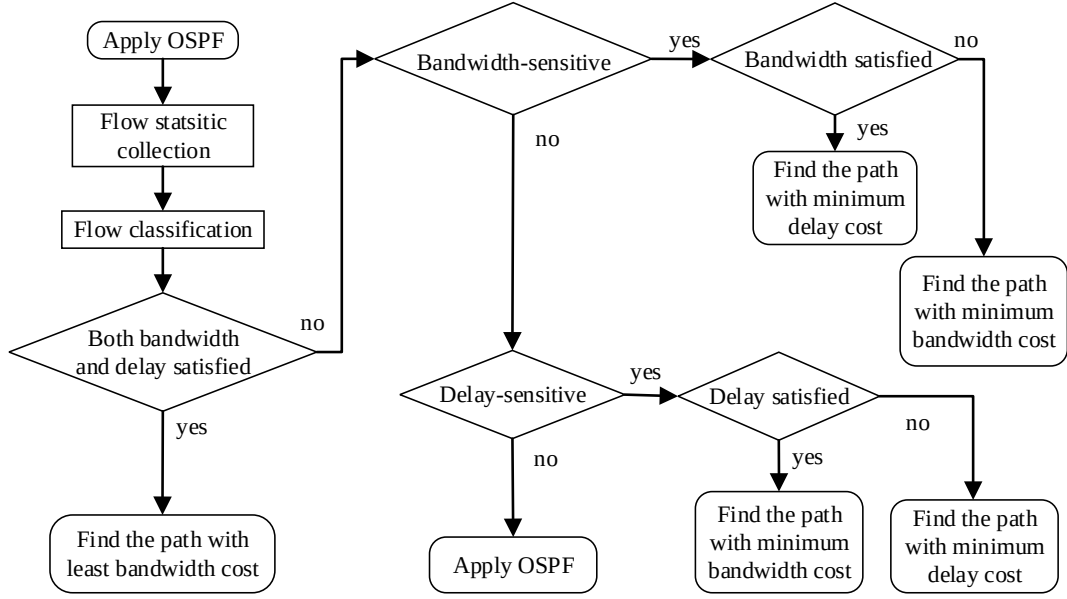


Figure 3 QoS-aware routing algorithm

Initially, when a packet is received by an OFN, the OFN first checks its flow table entries. If no flow table entry is matched, this packet will be sent to the controller using *Packet-Out* message. After received the *Packet-Out* message, the controller first applies an OSPF [32] path to the packet and following packets that affiliated with the same flow. Simultaneously, the controller starts QoS-aware routing mechanism by collecting statistics information of the flow for a short period. Applying the flow classification mechanism, controller can classify the flow to a specific type, such as real-time video/audio/imaging, common data retrieval, or scalar sensor data. QoS requirements of each flow type are predefined to the controller following ITU standard definition [29]. Then, controller builds a table for the flow with six tuple information $\{flow\ ID, \{src, dst\}, flow\ type, required\ bandwidth, required\ delay, flag\}$ as shown in Table 2. *Flow ID* is used to represent the flow from same application with same QoS requirements. $\{src, dst\}$ pair denotes the connection from source to destination. *Flow type* is obtained after flow classification as the basis to specify the QoS requirements. Flow's QoS requirements are the predefined values for each flow type. *Flag* is set to identify whether the flow is bandwidth sensitive, delay sensitive, or best-effort traffic. Best-effort traffic are the flows that prefer higher bandwidth and delay tolerant, such as data backup from source to a data storage (like FTP).

Table 1 Flow classification

flow ID	{src dst}	flow type	flow's QoS requirements		flag
			bandwidth	delay	
f1	(s_1, t_1)	real-time audio	128Kbps	<100ms	Delay-sensitive
f2	(s_2, t_2)	real-time video	5Mbps	<150ms	Bandwidth-sensitive & delay-sensitive
f3	(s_3, t_3)	Best-effort	high	-	delay tolerant

With the $\{src, dst\}$ pair, available paths from source to destination are ranked depending on bandwidth, delay, and utilization rate of the path. Comparing with flow's QoS requirements, two cases may happen, (1) feasible paths exist, (2) feasible path doesn't exist. This two cases are explained as two steps to find an optimal path.

Step 1: finding a feasible path satisfying QoS requirements of the flow

Feasible paths should satisfy flow's bandwidth and delay constraints. Result of Eq. (6) gives the bandwidth cost of each path for each flow. Pseudo code is shown as Algorithm 1. The path that can satisfy flow's bandwidth and delay and has the lowest cost is selected as the optimal path.

Algorithm 1: finding a feasible path
 Variables: n : number of available paths, b_{req} : required bandwidth,
 d_{req} : required delay, b_{pai} : available bandwidth of path i , p_i : the i -th
 path, $cost_{bpi}$: bandwidth cost of path i .

given a session pair
Loop
 for ($i = 0, i \leq n, i++$)
 if ($b_{req} \leq b_{pai} \ \&\& \ d_{req} \geq d_{pi}$)
 if ($cost_{bpi} > cost_{bpi+1}$)
 $p = p_{i+1}$
 $i++$;
 end *Loop*

If there is no feasible path found, the routing algorithm will go to Step 2 to find a best-effort path on QoS satisfaction.

Step 2: finding an optimal path when no feasible path exists

Due to the instability and the resource restriction of WSNs, more attentions should be paid to the case that no path satisfies the flow's QoS requirements. In this case, the algorithm is divided into three cases depending on the categories of the flow: bandwidth sensitive, delay sensitive, and best-effort, in order to supply QoS assurances to the flow as much as possible.

Case 1: routing algorithm for bandwidth-sensitive flows

Bandwidth satisfaction is more meaningful than delay for many applications. For instance, bandwidth guarantee for multimedia data retrieval from accident site could provide smooth and high definition video/images. For bandwidth-sensitive flows, priority of bandwidth is much higher than latency. If there are paths satisfying bandwidth requirement while dissatisfying delay requirement, the path with lowest delay cost is selected as the optimal path. Delay cost is defined as hop counts of the path by Eq. (7) (8). In this situation, among bandwidth satisfied paths, the one that has smallest hop count is selected. If there is no path that can satisfy bandwidth requirement, the path with largest available bandwidth will be selected.

$$\begin{cases} cost_{de_i} = 1, (e_i \in p) \\ cost_{de} = 0, \text{ otherwise} \end{cases} \quad (7)$$

$$cost_{dp} = \sum_{i=1}^n cost_{de_i} \quad (8)$$

Algorithm 2 illustrates the path selection process for bandwidth-sensitive flows when no feasible path found.

Algorithm 2: finding a path for bandwidth-sensitive flows
Variables: n : number of available paths, b_{req} : required bandwidth,
 d_{req} : required delay, b_{pai} : available bandwidth of path i , p_i : the i -th
path, $cost_{dpi}$: delay cost of path i .

given a session pair
Loop
for ($i = 0, i \leq n, i++$)
if ($b_{req} \leq b_{pai} \ \&\& \ d_{req} \leq d_{pi}$)
if ($cost_{dpi} > cost_{dpi+1}$)
 $p = p_{i+1}$
else if ($b_{req} \geq b_{pai}$)
if ($b_{pai} < b_{pai+1}$)
 $p = p_{i+1}$
 $i++$;
end *Loop*

Case 2: routing algorithm for delay-sensitive flows

For delay-sensitive flows, delay has more impact on data transmission performance than bandwidth. Therefore, if there are paths satisfy delay requirement but dissatisfy bandwidth requirement, the path with lowest bandwidth cost will be selected as the optimal path. If there is no path satisfies the delay requirement, the path with lowest delay will be selected. The routing process is shown in Algorithm 3.

Algorithm 3: finding a path for delay-sensitive flows
Variables: n : number of available paths, b_{req} : required bandwidth,
 d_{req} : required delay, d_{pi} : delay of path i , p_i : the i -th path, $cost_{dpi}$:
delay cost of path i .

given a session pair
Loop
for ($i = 0, i \leq n, i++$)
if ($d_{req} \leq d_{pi} \ \&\& \ b_{req} \geq b_{pi}$)
if ($cost_{bpi} > cost_{bpi+1}$)
 $p = p_{i+1}$
else if ($d_{req} \geq d_{pi}$)
if ($d_{pi} > d_{pi+1}$)
 $p = p_{i+1}$
 $i++$;
end *Loop*

Case 3: routing algorithm for best-effort flows

Best-effort flows are non-detrimental traffic that doesn't sensitive to QoS metrics. Typical applications would be peer-to-peer file sharing, historical data retrieval, and data backup. Best-effort flows usually has lowest priority, so that network resource should be allocated to other QoS sensitive applications preferentially. The residual network resource can be used for data transmission of best-effort applications. If new applications with higher priority injected to the network, best-effort flows should be confined to release enough resource for QoS assurance of the new applications.

4. Experiments and Performance Evaluation

There are some SDN/OpenFlow simulators, such as Mininet [33] and EstiNet [34]. And there are some well-known simulators for routing protocols, such as NS2 [35] and OPNET [36]. The problem is that SDN/OpenFlow simulators don't support the implementation of new routing mechanisms, while well-known routing simulators usually are not compatible to OpenFlow protocol. Therefore, we built a physical network testbed using virtual machines (VMs) and *Open vSwitch* software [37] installed at OFNs. Three typical flow types (video, audio, and file sharing) were selected to evaluate the performance of proposed QoS-aware routing mechanism. The comparison was conducted with traditional routing algorithms, OSPF and RIP.

4.1 Experiment Environment

As shown in Fig. 4, the overlay network testbed was made up of *OFNs*, *Source*, *Sink*, and *Controller*. *OFNs* were built using *Open vSwitch* software (version 2.0.2). Ubuntu servers (14.0.4.4 LTS) deployed with VLC server, VLC client, and *Ryu controller* (Release 1.0) [38] in VMs using Virtualbox (version 5.0.10) act as the *Source*, *Sink*, and *Controller* respectively. The software Quagga [39] was used to build Linux Routers for the implementation of routing protocol RIP and OSPF with the same network topology. Delay and bandwidth of each link was set by Linux command *tc*.

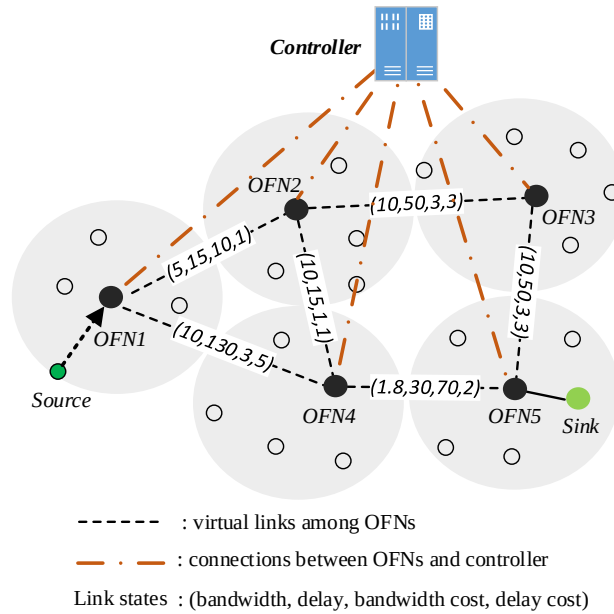


Figure 4 Testbed for OpenFlow-enabled WMSN

4.2 Methodology

When a sensor node (*Source* in Fig. 4) starts a new flow to *Sink*, *OFN1* will receive the

first packet of the flow. Without previously installed rules for the incoming flow, *OFN1* forwards the first packet to the controller. If the controller cannot determine the flow type, it applies an OSPF path to the flow and observes the flow statistics information. After the success of flow classification, the controller assigns a new path to the flow according to the proposed routing algorithms.

We tested the performance of RIP, OSPF and proposed QoS-aware routing mechanism for real-time video (bandwidth-sensitive & delay-sensitive), audio (delay-sensitive), and file sharing (best-effort traffic). Real-time video flow was emulated by 3Mbps streaming traffic from Source to Sink using *VLC player*. Real-time audio flow was emulated by 100Kbps UDP traffic using *iperf*, where only delay was evaluated since the available bandwidth of any link can satisfy the bandwidth requirement. File sharing was emulated by TCP traffic with buffer length 2Mbps, window size 56Kbytes, and max segment size 512 bytes. The link states are indicated in Fig. 3 with bandwidth, delay, bandwidth cost and delay cost.

In the experiments, all traffic went through path (*Source, OFN1, OFN4, OFN5, Sink*) for RIP, which has the smallest hop counts. QoS metrics of the path is (1.8, 160, 73, 7). Path (*Source, OFN1, OFN4, OFN2, OFN3, OFN5, Sink*) with QoS metrics (10, 245, 10, 14) was selected as the default path of OSPF, which has the highest bandwidth. Paths for different flows were different for QoS-aware routing mechanism depending on the comparison between flow's QoS requirements and network conditions. Take 3Mbps real-time video streaming as an example, path (*Source, OFN1, OFN2, OFN3, OFN5, Sink*) was determined as the route with QoS metrics (5, 115, 16, 9), which has higher bandwidth than RIP path and lower delay than OSPF path.

4.3 Tests and Evaluation Results

In the experiments, for bandwidth-sensitive and delay-sensitive video data transmission, both throughput and delay were evaluated. For delay-sensitive audio data transmission, only delay was evaluated. For best-effort file sharing, only bandwidth was compared since bandwidth leading to overall transmission speed.

4.3.1 Real-time Video Data Transmission

Video streaming server was put on *Source* and client was on *Sink*. Video data transmission through RIP, OSPF and QoS-aware routing paths was tested, as shown in Fig. 5, 6, 7. Bandwidth of RIP path is 1.8Mbps. To transmit 3Mbps streaming, 41% packet loss rate results in low video quality (Fig. 5). Bandwidth of OSPF path is adequate for the streaming but delay is around 248ms which is longer than requirement (150ms). Thereby in Fig. 6, the frames of both sides reveal apparent time difference, which may result in low quality of experience for interactive applications. Streaming through QoS-aware routing path is displayed in Fig. 7, where both frames are nearly same with no packet loss and small one-way delay (115ms).

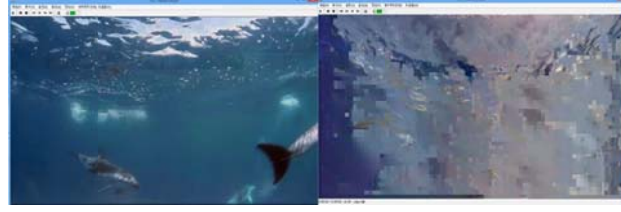


Figure 5 Video quality through RIP path

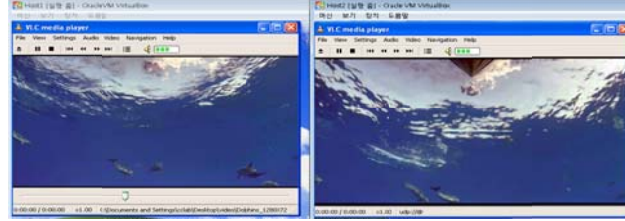


Figure 6 Video quality through OSPF path

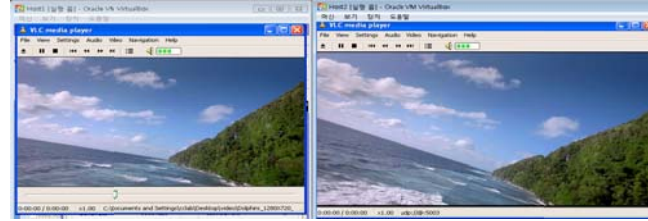


Figure 7 Video quality through QoS-aware path

We also tested the throughput variation when the path switched from RIP path to QoS-aware routing path. The throughput is significantly increased from around 1.73Mbps to 3Mbps, and jitter is reduced from around 6ms to 1ms (Fig. 8). Consequently, quality of video data transmission through QoS-aware routing path is much better than RIP path and OSPF path.

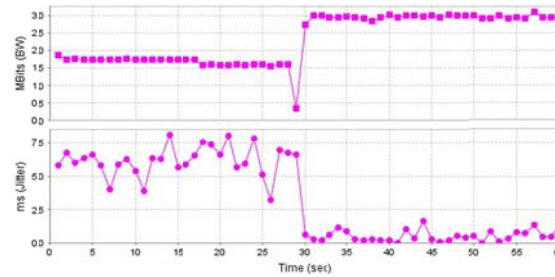


Figure 8 Path switching from RIP path to QoS-aware path

4.3.2 Real-time Audio Data Transmission

Generally, packet size of audio data is small and data rate is much lower than video. Transmission of audio data requires little bandwidth but sensitive to delay. Therefore, we emulate audio streaming by 100Kbps UDP traffic with packet size 300bytes. Test results in Fig. 9 indicate that the average delay is 165ms for RIP path, 249ms for OSPF path while 63ms for QoS-aware routing path respectively. According to the definition in Table 1, only the QoS-aware path can satisfy the delay requirement (100ms).

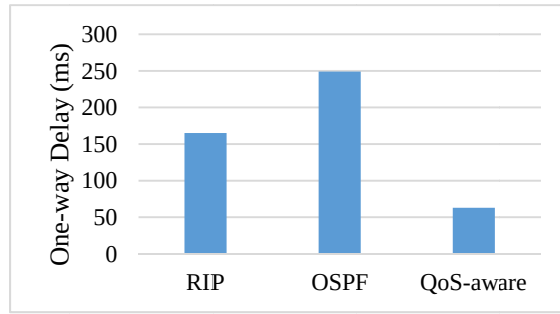


Figure 9 Delay test for audio data transmission

4.3.3 File Sharing Data Transmission

File sharing applications have lower priority than other applications. It desires higher bandwidth for fast data transmission. Therefore, the path with highest available bandwidth is usually assigned to those applications. In our scenario, bandwidth of RIP path is 1.8Mbps while OSPF path is 10Mbps. Without regard to other flows, QoS-aware routing path is the same path with OSPF. Test result indicates that when path switching from RIP path to QoS-aware routing path, throughput increases 5.5 times from 1.73Mbps to 9.64Mbps as shown in Fig. 10. Within 60 seconds, the transmitted data size is 12.4Mbytes through RIP path while 68.8Mbytes through QoS-aware routing path.

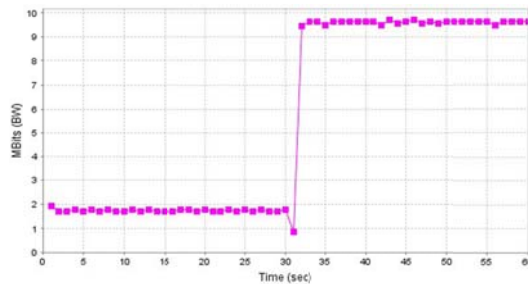


Figure 10 Performance improvement from RIP path to QoS-aware path

4.3.4 Summary

Performance evaluation was conducted with implementations of OSPF, RIP and proposed QoS-aware routing mechanism in a SDN testbed. Experiment results show that QoS-aware routing mechanism increased the throughput by 43% than RIP for video data transmission, and reduced the delay by 30% and 54% than RIP and OSPF for audio data transmission respectively. QoS-aware routing algorithm has advantages in QoS satisfaction for different types of flows. The advantages are derived from adopting SDN technology, which allows us to decide per-flow routing policy to mostly satisfy the QoS requirements.

5. Conclusion

In today's wireless sensor networks, more and more complex applications are deployed

and QoS-sensitive multimedia data is growing in the network. The SDN technology allows us to find the optimal path that can provide per-flow QoS assurance as much as possible. In this paper, we proposed a QoS-aware routing mechanism for OpenFlow-enabled WMSNs, which includes a QoS routing framework and algorithms. The framework, located at the application layer of SDN architecture, is composed of five components: link state detection, flow classification, flow management, QoS-aware routing algorithms, and per-flow routing policy. The routing algorithms seek feasible paths that can satisfy flow's QoS requirements, if exist. Otherwise, it decides optimal path depending on the flow types: delay-sensitive, bandwidth-sensitive, and best-effort traffic.

We built an SDN overlay network testbed made up of OFNs using *Open vSwitch* and SDN controller using *Ryu Controller*. A series of experiments were conducted to compare the routing performance for multimedia data transmission. The result shows that the proposed routing algorithm has overall advantage in QoS satisfaction for different types of flows.

This mechanism can be widely applied to WMSN applications, such as telemedicine, inter-vehicle communications, and environmental monitoring. In the future, we will further investigate to introduce other QoS parameters into QoS metrics, such as packet loss, reliability, and node power level. We will also take sensor node mobility into consideration on routing decision.

Conflict of Interests

The authors declare that there is no conflict of interests regarding the publication of this manuscript.

Acknowledgement

This work was supported by ICT R&D program of MSIP/IITP, Republic of Korea [NO.B0126151078-0002003, Creation of PEP based on automatic protocol behavior analysis and Resource management for hyper connected for IoT Services].

References

- [1] I. F. Akyildiz, T. Melodia, and K. R. Chowdhury, "A survey on wireless multimedia sensor networks," *Computer Networks*, October 2006.
- [2] J. Paek, J. Hicks, S. Coe, and R. Govindan, "Image-Based Environmental Monitoring Sensor Application Using an Embedded Wireless Sensor Network," *Sensors*, vol. 14, no. 9, pp. 15981-16002, Aug. 2014.
- [3] R. Bhatt and R. Datta, "A two-tier strategy for priority based critical event surveillance with wireless multimedia sensors," *Wireless Networks*, vol. 22, no. 1, pp. 267-284, Jan. 2016.
- [4] A. A. A. Youssif, A. Z. Ghalwash, and M. E. E. Dien, "ACWSN: an adaptive cross layer framework for video transmission over wireless sensor networks," *Wireless Networks*,

- vol. 21, no. 8, pp. 2693-2710, Nov. 2015.
- [5] R. Bruno and M. Nurchis, "Efficient data collection in multimedia vehicular sensing platforms," *Pervasive and Mobile Computing*, vol. 16, pp. 78-95, Jan. 2015.
 - [6] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114-119, February 2013.
 - [7] D. Kreutz, F.M.V. Ramos, P. Esteves Verissimo, C. Esteve Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," in *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14-76, January 2015.
 - [8] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69-74, April 2008.
 - [9] A. Mahmud and R. Rahmani, "Exploitation of OpenFlow in wireless sensor networks," in *Proceedings of International Conference on Computer Science and Network Technology (ICCSNT)*, pp. 594-600, Harbin, China, December 2011.
 - [10] T. Luo, H.P. Tan, and T.Q.S. Quek, "Sensor OpenFlow: enable software-defined wireless sensor networks," *IEEE Communications Letters*, vol. 16, no. 11, pp. 1896-1899, 2012.
 - [11] S. Costanzo, L. Galluccio, G. Morabito, and S. Palazzo, "Software defined wireless networks: Unbridling SDNs," in *Proceedings of European Workshop on Software Defined Networking (EWSDN)*, pp. 1-6, Darmstadt, Germany, October 2012.
 - [12] Z. Xia, Y. Hong-Yi, and Z. Gang, "Bandwidth efficient collaborative quality of service routing for real-time flow in wireless multimedia sensor networks," in *Proceedings of the IEEE Asia-Pacific Services Computing Conference (APSCC)*, pp. 509-515, Hangzhou, China, December 2010.
 - [13] F. Zabin, S. Misra, I. Woungang, H. F. Rashvand, N.-W. Ma, and M. A. Ali, "REEP: Data-centric, energy-efficient and reliable routing protocol for wireless sensor networks," *IET Communications*, vol. 2, no. 8, pp. 995-1008, 2008.
 - [14] A. H. Mohajerzadeh, M. H. Yaghmaee, N. N. Toroghi, S. Parvizy, and A. H. Torshizi, "MREEP: A QoS based routing protocol for wireless multimedia sensor networks," in *Proceedings of the IEEE 19th Iranian Conference on Electrical Engineering (ICEE '11)*, pp. 1-6, Tehran, Iran, May 2011.
 - [15] T. He, J. A. Stankovic, C. Lu, and T. Abdelzaher, "SPEED: A stateless protocol for real-time communication in sensor networks," in *Proceedings of the 23th IEEE International Conference on Distributed Computing Systems*, pp. 46-55, May 2003.
 - [16] V. Ukani, A. Kothari, and T. Zaveri, "An energy efficient routing protocol for wireless multimedia sensor network," in *Proceedings of the International Conference on Devices, Circuits and Communications*, pp. 1-6, Ranchi, India, September 2014.
 - [17] L. Han, S. Sun, B. Joo, and S. Han, "QoS-based path switching mechanism in mobile SCTP," *International Journal of Distributed Sensor Networks*, vol. 2014, Article ID 282181, 12 pages, 2014.
 - [18] R. Wu, "A novel cluster-based routing protocol in wireless multimedia sensor network," in *Proceedings of the 4th IEEE International Conference on Broadband Network and Multimedia Technology (IC-BNMT)*, pp. 126-129, Shenzhen, China, October 2011.
 - [19] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, "Energy-efficient

- communication protocol for wireless microsensor networks,” in *Proceedings of the 33rd Annual Hawaii International Conference on System Sciences*, January 2000.
- [20] S. H. Qin, J. Cao, J. N. Ye, and Y. P. Huang, “Real-time clustering routing protocol for multimedia sensor networks,” *Computer Engineering*, vol. 36, no. 17, pp. 129-131, 2010.
 - [21] G. Vithya and B. Vinayagasundaram, “Actuation sensor with adaptive routing and QoS aware checkpoint arrangement on wireless multimedia sensor network,” in *Proceedings of the International Conference on Recent Trends in Information Technology (ICRTIT)*, pp. 444-449, Chennai, India, June 2011.
 - [22] S. Sun, L. Han, and S. Han, “GRMR: Greedy regional multicast routing for wireless sensor networks,” *IEICE transactions on Information and Systems*, vol. E99-D, no. 1, pp. 21-29, January 2016.
 - [23] Z.-J. Han and W. Ren, “A novel wireless sensor networks structure based on the SDN,” *International Journal of Distributed Sensor Networks*, vol. 2014, Article ID 874047, 7 pages, 2014.
 - [24] K.-K. Yap, M. Kobayashi, R. Sherwood, N. Handigol, T. Y. Huang, M. Chan, and N. McKeown, “Openroads: Empowering research in mobile networks,” *SIGCOMM Computer Communication Review*, vol. 40, no. 1, pp. 125-126, January 2010.
 - [25] L. Ochoa-Aday, C. Cervelló-Pastor, and A. Fernández-Fernández, “Current trends of topology discovery in OpenFlow-based software defined networks,” *External research report*, September 2015.
 - [26] W. Braun and M. Menth, “Software-defined networking using OpenFlow: Protocols, applications and architectural design choices,” *Future Internet*, vol. 6, no. 2, pp. 302-336, May 2014.
 - [27] P. Phaal, S. Panchen, and N. McKee, “InMon Corporation’s sFlow: A method for monitoring traffic in switched and routed networks,” *IETF RFC 3176*, September 2001.
 - [28] V. Berk and J. Murphy, “Understanding Netflow monitoring, security, and forensics,” *Process Query Systems, LLC*, 2013.
 - [29] Y. Chen, T. Farley, and N. Ye, “QoS requirements of network applications on the Internet,” *Information Knowledge Systems Management*, vol. 4, no. 1, pp. 55-76, January 2004.
 - [30] A. Karim, I. Ahmad, S.I. Jami, and M. Sarwar, “Cluster analysis of traffic flows on a campus network,” in *Proceedings of the 24th IASTED international conference on Artificial intelligence and applications (ATA’06)*, pp. 416-421, Anaheim, CA, USA, 2006.
 - [31] B. Fortz, and M. Thorup, “Internet traffic engineering by optimizing OSPF weights,” in *Proceedings of the Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (INFOCOM)*, vol. 2, pp. 519-528, Tel-Aviv, Israel, March 2000.
 - [32] J. Moy, “OSPF version 2,” *IETF RFC 2328*, April 1998.
 - [33] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proceedings of the 9th ACM Workshop on Hot Topics in Networks*, Monterey, CA, USA, October 2010.
 - [34] S.-Y. Wang, C.-L. Chou, and C.-M. Yang, “Estinet openflow network simulator and emulator,” *IEEE Communications Magazine*, vol 51, no. 9, pp. 110-117, September

2013.

- [35] T. Issariyakul and E. Hossain, “Introduction to network simulator NS2,” *Springer Publishing Company, Incorporated*, 2010.
- [36] X. Chang, “Network simulations with OPNET,” in *Proceedings of the 31st conference on Winter simulation: Simulation---a bridge to the future (WSC’99)*, pp. 307-314, 1999.
- [37] B. Pfaff, et al., “The design and implementation of Open vSwitch,” in *the Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’15)*, pp. 117-130, Oakland, CA, USA, May 2015.
- [38] Ryu project team, “Ryu SDN framework, release 1.0,” <https://osrg.github.io/ryu-book/en/Ryubook.pdf>.
- [39] Quagga Routing Software Suite, GPL Licensed, <http://www.nongnu.org/quagga>.